

Introducing interfaces

We'll build upon our *Point* struct example by declaring a *Printer* interface, which declares a *Print* method:

```
type Printer interface {
    Print()
}

type Point struct {
    x, y int
}

func (p Point) Print() {
    fmt.Printf("x = %d, y = %d\n", p.x, p.y);
}

func main() {
    var p Printer = Point(10, 20)
    p.Print()
}
```

Interfaces are specified with the *interface* keyword. Note that these interfaces are not exactly the same as in C#/Java. The main difference is that a *struct* doesn't have to say it implements an interface—any *struct* that implements the methods specified by an interface satisfies that interface! Those familiar with dynamic languages will easily understand that it is similar to 'duck typing' in those languages. But remember Go is a strictly type-checking statically typed language, and it implements this feature!

A goroutines example

Goroutines are functions executing in parallel in the same address space in stack. They are not the same as threads or processes, so they get a new name, goroutines.

Goroutines communicate using 'channels' (based on CSP). Here is an example as given in the 'Effective Go' document (see the *Resources* section at the end of this article), which shows how a *Sort* on a big *list* can be done in parallel with some other computation.

```
c := make(chan int); // Allocate a channel.
// Start the sort in a goroutine; when it completes, signal on the channel.
go func() {
    list.Sort();
    c <- 1; // Send a signal; value does not matter.
})();
doSomethingForAwhile();
<-c; // Wait for sort to finish; discard sent value.
```

A 'channel' is for communication between goroutines; this code creates a channel using the *make* built-in function (which is similar to *new*, and not covered in this article). A goroutine is invoked using the *go* keyword. Here, the goroutine is created to call a function defined as a function literal, which does two things. First, it does a *Sort* on a *list* (assume that this is a time-consuming operation). Next, once the sorting is done,

it sends a 'signal' to the caller goroutine, that it is done with its work. Meanwhile, the caller code (which is a goroutine) does some other work in the call *doSomethingForAwhile*, and waits for the other goroutine to finish in the statement "*<-c*,". This code segment effectively shows how goroutines are created and how communication is done between them.

Implementation status

Currently, Go implementations are available for two platforms: Linux and Mac OS X. There are two implementations for these platforms: one is a stand-alone implementation of a Go compiler and runtime, written by Ken Thompson. Another is a GCC front-end, implemented by Ian Lance Taylor. What about Windows? Some unofficial ports exist, but they are not very stable or up-to-date. I installed Go in Fedora, and the installation was smooth and easy.

All of Go's toolset (compilers, packages, runtime libraries, tools, etc) is available as open source, under a BSD licence.

Wrapping up

With my experience in trying Go over the last few months and writing non-trivial programs, I think there is a lot of substance behind the hype. Go certainly has some unique features that will make it a useful systems programming language, in this age of the Internet and multi-core processors. In just a couple of days since I started learning Go, I was able to write non-trivial programs, and began liking the language. It was fun learning and playing around with it. I would recommend Go to any systems programmer and encourage them to try it out. 

Resources

Go websites:

- The official website is www.golang.org (the Web server is implemented in Go!). Lots of material on Go is available from this website, including tutorials and presentations. You can download Go compilers and tools from this website. An unofficial Go website is at <http://go-lang.cat-v.org/>— it has links to download the Windows version of Go, and has links to numerous Go-related resources.

Learning Go:

- As of this writing, there is no book available on Go. However, the Go website has very good documents for learning Go. Check 'Effective Go' at http://golang.org/doc/effective_go.html. Robert Pike's talk introducing Go is available at <http://www.youtube.com/watch?v=rKnDgT73v8s>. Go's discussion group is at <http://groups.google.com/group/golang-nuts/>.

Try Go online:

- I find it really handy to compile and run Go programs online (!) at the IDEOne website <http://ideone.com/>

By: S.G. Ganesh

The author is a research engineer in Siemens (Corporate Technology), Bangalore. His latest book is a best seller—'Cracking the C, C++ and Java Interview', published by Tata McGraw-Hill, New Delhi. You can reach him at sgganesh@gmail.com.